CSE-Tutor: Verified Synthetic Data for Fine-Tuning a 1.5B-Parameter Tutor Model for Computer Science Education

Satya Dedeepya

Course project, Deep Learning · github.com/satyadedeepya/slm_dl_project

***Abstract*—Small language models can run on a student's laptop, but they are unreliable on the numerical problems that dominate computer-science examinations. We fine-tune Qwen2.5-1.5B-Instruct with LoRA as a tutor for six subjects (data structures and algorithms, programming, databases, operating systems, computer networks and deep learning) using a mixed instruction set whose distinctive component is *programmatically verified* data: 3,000 problems whose step-by-step solutions and answers are computed by executing the underlying algorithm. All training and evaluation runs on an integrated Intel Arc GPU. On held-out test sets, fine-tuning significantly improves text-to-SQL execution accuracy (50.5→60.0), functional-dependency problems (15→30) and network numericals (1→8) over the base model (exact McNemar $p < 0.05$), without loss on code generation. An ablation that removes only the verified data eliminates these numerical gains. A continued-training stage adds deep learning, which raises value-level accuracy on held-out deep-learning numericals from 15.8% to 26.0% (paired permutation $p = 0.013$) while every earlier gain is retained. Error analysis shows that the remaining failures are mostly procedural: the model reproduces the solution template but executes multi-step algorithms unreliably.**

***Index Terms*—small language models, LoRA, synthetic data, data verification, computer science education, text-to-SQL.**

I. Introduction

Students preparing for university examinations, GATE and placement interviews need an assistant that explains concepts, solves numerical problems step by step and writes correct code and SQL. Large hosted models do this well but cost money and need connectivity; a 1.5B-parameter model runs locally, but out of the box it computes CPU-scheduling waiting times, page faults and subnet ranges incorrectly because it pattern-matches instead of executing the procedure.

The common recipe for adapting a small model is to fine-tune on question–answer pairs written by a larger model [11], [12]. Such labels are unverified, and a small student model can learn the teacher's mistakes. We take a different route for the part of the curriculum that is algorithmic: we write generators that sample random problem instances and *compute* their solutions by running the algorithm, so every label is correct by construction. We then ask three questions:

- **RQ1:** how much does LoRA fine-tuning improve a 1.5B model on CS tasks compared with zero- and few-shot prompting?
- **RQ2:** does the verified numerical data cause the improvement on numerical problems, or does the rest of the mix?
- **RQ3:** can a new subject (deep learning) be added cheaply by continued training without forgetting earlier skills?

Our contributions are (1) verified generators for 41 problem types across operating systems, networks, databases and deep learning, with tests against textbook examples and PyTorch; (2) a controlled ablation isolating the effect of verified data; (3) a statistically careful evaluation (bootstrap confidence intervals, exact McNemar tests, separated format failures) on seven held-out benchmarks; (4) an error taxonomy of the fine-tuned model; and (5) a local chat interface. The whole pipeline runs on a laptop with an integrated GPU.

II. Related Work

Parameter-efficient fine-tuning. LoRA [1] trains low-rank updates of frozen weights; QLoRA [2] adds 4-bit quantisation of the base model. We use LoRA on all attention and MLP projections of Qwen2.5-1.5B-Instruct [3], [4].

Models for CS education. GuideLM [6] shows that small supervised sets can shape pedagogical behaviour; QLoRA has been applied to DSA problems [7]; local open-source teaching assistants are studied in [8]–[10]. CS-Bench [5] is the closest benchmark and serves as our multiple-choice evaluation.

Synthetic instruction data. Self-Instruct [11] and GLAN [12] generate instruction data with language models. Our generators are programs, so correctness does not depend on a teacher model; LLM-written data in our mix is filtered by answer-key agreement instead.

Forgetting. Parameter-efficient tuning still forgets [13]; we therefore keep replay data and measure code generation and examination questions after every stage.

III. Data

A. Verified generators

Each generator samples parameters, runs the algorithm and writes a solution in a fixed teaching format that ends with a `Final answer:` line, plus a machine-readable grading specification (Table I). Questions state their conventions (tie-breaking, whether C-SCAN counts the return seek) so the reference answer is unambiguous. Test problems are drawn from parameter pools disjoint from training. Correctness is checked by unit tests against textbook worked examples (e.g. Silberschatz scheduling and page-replacement figures), property tests (Belady's optimality, safety-sequence validity), and, for deep learning, by recomputing answers with PyTorch layers and autograd.

TABLE I
VERIFIED GENERATOR FAMILIES

Subject	Families	Train / test
OS	scheduling (6 algorithms), page replacement, Banker's, disk scheduling	900 / 100
CN	subnetting, CRC, checksum, delays, sliding window	900 / 100
DBMS	closure, candidate keys, normal form, lossless join	600 / 100
DL	conv/pool shapes, MLP parameters, forward pass, softmax-CE, GD, backprop, momentum, attention, P/R/F1	600 / 100

B. The full training mix

The first-stage mix (9,335 examples, 2.68M tokens; Table II) combines the generators with GATE/ISRO previous-year questions whose explanations were produced by a teacher model *without the key* and kept only when the teacher agreed with the official answer (990 of 1,154), concept Q&A over a 133-topic syllabus, Q&A grounded in four openly licensed textbooks, Spider [14] text-to-SQL with real schemas, MBPP [15] and CodeAlpaca for code, and general replay data. All sources are deduplicated with MinHash and decontaminated against every evaluation set with 13-gram overlap. GATE papers up to 2020 and half of 2021–2026 are used for training; the other half of 2021–2026 is held out.

TABLE II
FIRST-STAGE TRAINING MIX (TRAIN + VALIDATION)

Source	Examples	Quality gate
Verified generators (OS, CN, DBMS)	2,400	computed answers
GATE / ISRO PYQs	978	teacher agrees with key
Concept Q&A (teacher)	1,458	MCQ/TF key agreement
Open-textbook Q&A	1,170	answerable from passage
Spider	1,335	gold SQL executes
MBPP / CodeAlpaca	311 / 1,184	tests pass / code parses
Replay (Alpaca)	499	length filter

IV. Method

Models. B0 and B1 are the base model prompted zero-shot and 3-shot. FT-B is LoRA on the full first-stage mix; FT-A is identical but without the generator slice (6,588 examples). FT-C continues training FT-B's adapter on a deep-learning stage: 600 verified DL problems, 307 DL concept Q&A items over 36 syllabus topics (teacher-generated, MCQ/TF filtered by key agreement) and 1,200 replay examples sampled from the first-stage mix (2,107 examples, 0.61M tokens, 66 steps).

Training. LoRA rank 16, $\alpha = 32$, dropout 0.05 on q, k, v, o, gate, up and down projections (18.5M trainable parameters, 1.18%); loss on assistant tokens only; AdamW, learning rate 2×10^{-4} (10^{-4} for FT-C) with cosine decay and 3% warm-up; effective batch 32 (batch 1 × 32 accumulation); one epoch; bf16; length-grouped batches.

Hardware. An Intel Core Ultra 5 125H with integrated Arc graphics (8.4 GB shared memory). bitsandbytes 4-bit quantisation is not dependable on Intel XPU, so the base model is held in bf16 (the adapter is the same as in QLoRA). Batch 4 ran out of memory because the 152K-token vocabulary makes the logits alone about 2.5 GB, hence gradient accumulation. One optimiser step takes about 62 s; FT-B trained in 4.8 h and FT-A in 3.7 h.

Evaluation. Adapters are merged and decoded greedily with the prompts used for the baselines: CS-Bench [5] (400 DSA/OS/CN multiple-choice and true/false items, accuracy), verified OS/CN/DBMS/DL test problems (100 each, exact match by a typed grader), MBPP (200, pass@1 with test execution), Spider dev (200, execution accuracy) and the held-out GATE CS questions (56). We report 95% bootstrap confidence intervals and exact two-sided McNemar tests on paired items, and count unparseable answers separately so that learning the answer format is not mistaken for knowledge.

V. Results

TABLE III
ACCURACY (%) ON HELD-OUT TASKS. P: EXACT MCNEMAR TEST AGAINST B0 (FT-B, FT-A) OR AGAINST FT-A (FT-B VS FT-A)

Task	n	B0 0-shot	B1 3-shot	FT-A	FT-B	FT-C	p FT-B vs B0	p FT-B vs FT-A
CS-Bench (DSA/OS/CN)	400	51.2	**58.0**	53.8	55.2	–	0.18	0.57
OS numericals	100	4.0	4.0	3.0	5.0	–	1.00	0.73
CN numericals	100	1.0	2.0	0.0	**8.0**	7.0	**0.039**	**0.008**
DBMS FD problems	100	15.0	12.0	17.0	**30.0**	**31.0**	**0.017**	0.060
DL numericals	100	2.0	–	–	3.0	**5.0**	0.45†	–
MBPP pass@1	200	41.5	41.5	41.0	42.5	40.0	0.89	0.66
Spider (execution)	200	50.5	–	54.5	**60.0**	59.5	**0.011**	0.052
GATE CS 2021–26	56	33.9	–	37.5	33.9	–	1.00	0.82

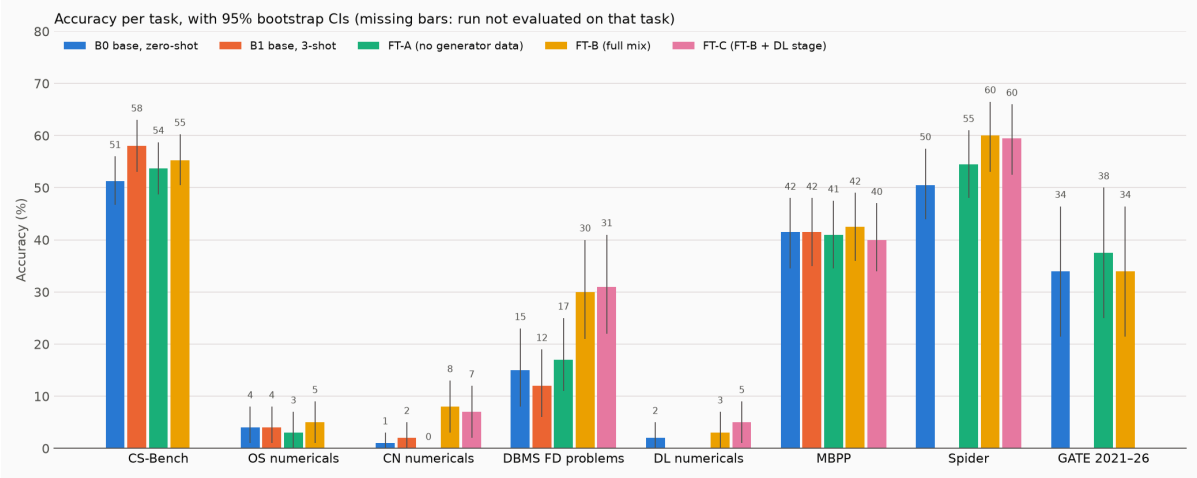


Fig. 1. Accuracy per task with 95% bootstrap confidence intervals. Missing bars: run not evaluated on that task (FT-C was evaluated on the five most informative tasks).

A. RQ1: fine-tuning versus prompting

FT-B significantly improves over the zero-shot base model on Spider (+9.5 points, $p = 0.011$), DBMS functional-dependency problems (+15.0, $p = 0.017$) and network numericals (+7.0, $p = 0.039$). On CS-Bench the gain (+4.0) is not significant, and about half of it comes from fewer unparseable answers (21→7 of 400); three-shot prompting remains slightly better on this knowledge-oriented benchmark. MBPP and GATE do not change significantly, so first-stage fine-tuning does not degrade code generation or examination performance. The only negative signal is CS-Bench's *reasoning* subset, which fell by 6–7 points in both fine-tuned runs (not significant at $n = 142$).

B. RQ2: what the verified data contributes

FT-A, trained on the same mix and settings minus the generator slice, stays at base-model level on every numerical task (CN 0%; closure and candidate keys 0% versus 32.8% for FT-B). The numerical gains are therefore caused by the verified data rather than by teacher Q&A, textbooks or examination questions. On CS-Bench, MBPP and GATE the two runs are within noise: the generated data is specific in its effect. FT-B also trains for 35% more steps than FT-A, so part of any FT-B–FT-A difference could come from training length; a step-matched control is left to future work. CPU and disk scheduling stay at 0% for every model: FT-B reproduces Gantt charts and tables perfectly but fills them with wrong values.

C. RQ3: adding deep learning by continued training

The deep-learning stage took 70 min of training (66 steps). On the 100 held-out DL problems FT-C reaches 5.0% exact match, against 3.0% for FT-B and 2.0% for the base model (†McNemar FT-C vs B0). Exact match is a harsh metric here because every number must be right (an output shape, a parameter count and a bias all at once). Measured per expected value, with the grader's tolerance, FT-C produces 26.0% of the correct values, against 15.8% for FT-B and 12.6% for the base model (Table V); the improvement over FT-B is significant under a paired sign-flip permutation test ($p = 0.013$; $p = 0.001$ against the base model). Per problem type, value-level accuracy rises most for convolution shapes (22→47%), softmax/cross-entropy (0→56%) and classification metrics (22→44%), while one-step gradient descent (36→8%) and momentum (31→19%) get worse and attention weights and sigmoid backpropagation stay at 0%. In the convolution problems FT-C applies the right formulas ($\lfloor (N + 2P - K)/S \rfloor + 1$ and $(K \cdot K \cdot C + 1) \cdot F$) but often slips on multi-digit products, the same procedural pattern as in Section VI.

The stage did not erode earlier skills: against FT-B, DBMS (31.0 vs 30.0), CN (7.0 vs 8.0), Spider (59.5 vs 60.0) and MBPP (40.0 vs 42.5) all differ by at most a few items (McNemar $p \geq 0.38$), and FT-C keeps its significant gains over the base model on DBMS ($p = 0.003$) and Spider ($p = 0.015$). Mixing 1,200 replay examples into a 2,107-example stage was enough to prevent forgetting at this scale.

TABLE V
DL NUMERICALS (N = 100): EXACT MATCH AND VALUE-LEVEL ACCURACY (%)

Model	Exact	Values [95% CI]	p vs prev.
B0 base	2.0	12.6 [8.3–17.2]	–
FT-B	3.0	15.8 [11.0–20.8]	0.27
FT-C	**5.0**	**26.0** [19.8–32.2]	**0.013**

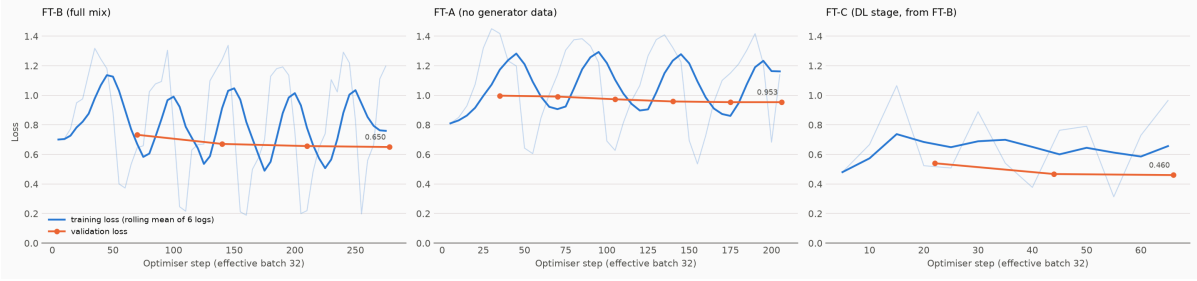


Fig. 2. Training loss (rolling mean) and validation loss. The runs use different validation samples and are not directly comparable.

VI. Error Analysis

We hand-labelled a stratified random sample of 50 wrong FT-B answers (12 CS-Bench, 8 each OS/CN/DBMS, 7 MBPP, 7 Spider) with one primary cause each (Table IV). None was a format failure.

TABLE IV
ERROR CATEGORIES OF FT-B (N = 50)

Category	Share	Typical case
Procedure / state tracking	52%	closure stops after one FD
Knowledge gap	16%	avoidance vs prevention
Schema hallucination	12%	column from wrong table
Arithmetic / units	8%	0.96 ms written as 9.6 ms
Misread task	6%	number returned, bool expected
Answer contradicts reasoning	4%	argues A, answers B
Ambiguous item	2%	–

Procedure errors account for 20 of the 24 numerical failures and fall into three patterns: *early stopping in fixed-point loops* (attribute closure applies one dependency and declares that nothing else applies), *unfaithful reading of the input* (claiming an attribute never appears on a right-hand side when it does; ordering FCFS by process number instead of arrival) and *inconsistent state* (LRU tables wider than the frame count; negative waiting times that are never checked). Arithmetic slips are a minority and are mostly powers of ten. Spider failures are grounding errors against a schema that is in the prompt. These patterns suggest concrete remedies: traces that restate the complete state at each step with built-in self-checks, tool use for arithmetic, and execution feedback for SQL.

VII. System

The tutor is served by a small FastAPI application that loads the base model once and attaches the FT-A, FT-B and FT-C adapters as named LoRA adapters, so the chat interface can switch between them, or disable them to show the base model, without reloading weights. Answers stream token by token with server-sent events; the browser interface renders Markdown, tables and highlighted code and keeps conversations locally. A second Gradio interface generates a fresh verified problem, lets the model solve it and grades the answer automatically. On the integrated GPU a typical answer streams in 20–30 s.

VIII. Limitations

- One epoch per run and one seed per configuration; validation loss was still falling slowly.
- FT-B trains for more steps than FT-A, which partly confounds the ablation.
- The GATE test has only 56 items (±12-point intervals); MMLU was not evaluated.
- Error labels come from a single annotator on a quota-stratified sample.
- FT-C was evaluated on five of the eight tasks (DL, DBMS, CN, Spider, MBPP) to save time; CS-Bench, OS and GATE were not re-run.

IX. Conclusion

Verified, programmatically generated training data teaches a 1.5B model procedures that neither prompting nor LLM-written data teach: functional-dependency closure and keys, subnetting and CRC; a short continued-training stage with verified problems and replay adds measurable deep-learning skill (value-level 15.8→26.0%) without forgetting. The gains are specific and do not cost code-generation ability. The main limitation is execution of long procedures, which the model imitates rather than performs. Future work: more epochs and self-checking traces, tool use for arithmetic and simulation, preference tuning on verified correct/incorrect pairs, and a step-matched ablation.

References

1. E. J. Hu *et al.*, “LoRA: Low-rank adaptation of large language models,” arXiv:2106.09685, 2021.
2. T. Dettmers *et al.*, “QLoRA: Efficient finetuning of quantized LLMs,” in *NeurIPS*, 2023.
3. Qwen Team, “Qwen2.5 technical report,” arXiv:2412.15115, 2024.
4. B. Hui *et al.*, “Qwen2.5-Coder technical report,” arXiv:2409.12186, 2024.
5. X. Song *et al.*, “CS-Bench: A comprehensive benchmark for large language models towards computer science mastery,” arXiv:2406.08587, 2024.
6. “Supervised fine-tuning LLMs to behave as pedagogical agents in programming education,” arXiv:2502.20527, 2025.
7. “Fine-tuning of large language models for solving DSA problems using QLoRA,” IEEE, doc. 10969969.
8. “AI-TA: Towards an intelligent question-answer teaching assistant using open-source LLMs,” arXiv:2311.02775, 2023.
9. “Small models, big support: A local LLM framework for educator-centric content creation and assessment,” arXiv:2506.05925, 2025.
10. “A large-scale real-world evaluation of LLM-based virtual teaching assistant,” arXiv:2506.17363, 2025.
11. Y. Wang *et al.*, “Self-Instruct: Aligning language models with self-generated instructions,” arXiv:2212.10560, 2022.
12. H. Li *et al.*, “Synthetic data (almost) from scratch: Generalized instruction tuning for language models,” arXiv:2402.13064, 2024.
13. “Analyzing and reducing catastrophic forgetting in parameter efficient tuning,” arXiv:2402.18865, 2024.
14. T. Yu *et al.*, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *EMNLP*, 2018.
15. J. Austin *et al.*, “Program synthesis with large language models,” arXiv:2108.07732, 2021.